

Notes on Sarsa, n -step Sarsa, and Monte Carlo Control

1 Setting

As before, we work in the usual MDP framework: states $s \in \mathcal{S}$, actions $a \in \mathcal{A}$, rewards $r \in \mathbb{R}$, discount factor $\gamma \in [0, 1]$, and a (possibly stochastic) policy $\pi(a \mid s)$. The action-value function under π is

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right].$$

Sarsa and Monte Carlo control are both *on-policy*: the policy generating the data is the same one being evaluated and improved. They differ in how far they look before updating $\hat{Q}$ — this is the temporal-difference (TD) vs. Monte Carlo (MC) distinction, and n -step Sarsa is the bridge between the two.

2 Sarsa: one-step TD control

Sarsa updates $\hat{Q}$ after every single transition, using the actually observed next action (hence *on-policy*, and hence the name: State, Action, Reward, State, Action):

$$\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha \left(R_{t+1} + \gamma \hat{Q}(S_{t+1}, A_{t+1}) - \hat{Q}(S_t, A_t) \right).$$

The quantity $R_{t+1} + \gamma \hat{Q}(S_{t+1}, A_{t+1})$ is the **TD target**, and the bracketed term is the **TD error** δ_t . Unlike MC, this target *bootstraps*: it uses the current estimate $\hat{Q}(S_{t+1}, A_{t+1})$ instead of waiting for the true return, so it is biased (initially) but has much lower variance and does not require the episode to end.

Algorithm 1 Sarsa (on-policy TD control)

```
1: Initialize  $\hat{Q}(s, a) \leftarrow 0$  for all  $s, a$  (terminal states: 0)
2: for episode = 1, 2, ... do
3:   Initialize  $S_0$ ; choose  $A_0$   $\varepsilon$ -greedily from  $\hat{Q}(S_0, \cdot)$ 
4:   for  $t = 0, 1, 2, \dots$  until  $S_t$  terminal do
5:     Take  $A_t$ , observe  $R_{t+1}, S_{t+1}$ 
6:     Choose  $A_{t+1}$   $\varepsilon$ -greedily from  $\hat{Q}(S_{t+1}, \cdot)$ 
7:      $\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha (R_{t+1} + \gamma \hat{Q}(S_{t+1}, A_{t+1}) - \hat{Q}(S_t, A_t))$ 
8:   end for
9: end for
```

Because A_{t+1} is chosen by the same ε -greedy policy that is being improved, Sarsa's target depends on the exploratory action actually taken next. This is what makes it on-policy (contrast

with Q-learning, which bootstraps off $\max_{a'} \hat{Q}(S_{t+1}, a')$ regardless of what is actually done next — an off-policy target).

3 n -step Sarsa

One-step Sarsa bootstraps immediately (after a single reward); Monte Carlo (Section 4) waits for the entire episode. **n -step Sarsa** generalizes both: accumulate n real rewards, then bootstrap off $\hat{Q}$ at the n -th step.

3.1 The n -step return

Define the n -step return for the state-action pair at time t as

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{Q}(S_{t+n}, A_{t+n}) = \sum_{k=0}^{n-1} \gamma^k R_{t+k+1} + \gamma^n \hat{Q}(S_{t+n}, A_{t+n}),$$

with the usual truncation if the episode terminates before step $t+n$ (in that case the bootstrap term is dropped and $G_{t:t+n}$ is just the remaining discounted rewards — an ordinary MC-style return). The update rule is the same sample-average-style TD update, just with $G_{t:t+n}$ in place of the one-step target:

$$\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha (G_{t:t+n} - \hat{Q}(S_t, A_t)).$$

The bookkeeping (storing the last $n+1$ states/actions/rewards and delaying the update by n steps) is the only real complication versus one-step Sarsa; conceptually it is the identical ε -greedy GPI loop.

3.2 Bias–variance tradeoff in n

n interpolates between two extremes:

- $n = 1$: ordinary Sarsa. Low variance (only one random reward feeds the target), but biased while $\hat{Q}$ is inaccurate, and credit for a reward propagates back only one state per episode.
- $n = T - t$ (full episode length): equivalent to MC — the target is the actual return G_t , unbiased but high variance (it depends on every random reward and action along the rest of the episode).

Intermediate n (e.g. $n = 4$ or $n = 8$) often outperforms both extremes empirically: rewards propagate back n steps per update instead of 1, speeding up learning, while variance stays bounded because the tail beyond step n is replaced by the (lower-variance) bootstrap $\hat{Q}(S_{t+n}, A_{t+n})$ rather than sampled all the way to termination.

4 Monte Carlo control, revisited

Monte Carlo control fits the same $n \rightarrow \infty$ (i.e. $n = T$) end of the spectrum above: no bootstrapping at all, updates only after a complete episode, using the true observed return

Algorithm 2 n -step Sarsa (on-policy)

```
1: Initialize  $\hat{Q}(s, a) \leftarrow 0$  for all  $s, a$ ; fix step size  $\alpha$ , horizon  $n$ 
2: for episode = 1, 2, ... do
3:   Initialize and store  $S_0$ ; choose and store  $A_0$   $\varepsilon$ -greedily from  $\hat{Q}(S_0, \cdot)$ 
4:    $T \leftarrow \infty$ 
5:   for  $\tau = 0, 1, 2, \dots$  do
6:     if  $\tau < T$  then
7:       Take  $A_\tau$ , observe and store  $R_{\tau+1}, S_{\tau+1}$ 
8:       if  $S_{\tau+1}$  is terminal then
9:          $T \leftarrow \tau + 1$ 
10:      else
11:        Choose and store  $A_{\tau+1}$   $\varepsilon$ -greedily from  $\hat{Q}(S_{\tau+1}, \cdot)$ 
12:      end if
13:    end if
14:     $t \leftarrow \tau - n + 1$  ▷ time whose estimate is being updated
15:    if  $t \geq 0$  then
16:       $m \leftarrow \min(t + n, T)$ 
17:       $G \leftarrow \sum_{k=t+1}^m \gamma^{k-t-1} R_k$ 
18:      if  $t + n < T$  then
19:         $G \leftarrow G + \gamma^n \hat{Q}(S_{t+n}, A_{t+n})$ 
20:      end if
21:       $\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha(G - \hat{Q}(S_t, A_t))$ 
22:    end if
23:  end for
24:  until  $t = T - 1$ 
25: end for
```

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t-1} R_T.$$

(See the fuller treatment — first-visit vs. every-visit, sample-average vs. constant- α , GLIE — in the on-policy Monte Carlo notes in [on_policy_monte_carlo/notes/](#).)

4.1 Sarsa vs. n -step Sarsa vs. MC

	Sarsa ($n=1$)	n -step Sarsa	Monte Carlo
Bootstraps?	yes	yes (past step n)	no
Bias (early training)	high	moderate	none
Variance	low	moderate	high
Update frequency	every step	every step (delayed by n)	once per episode
Needs episode to terminate?	no	no	yes
Works in continuing tasks?	yes	yes	no

All three are instances of **Generalized Policy Iteration**: alternate ε -greedy improvement of π with evaluation of Q^π , the only difference being *how much real experience is collected per update* before the (possibly bootstrapped) target is formed.

Algorithm 3 On-policy MC control (constant- α , every-visit) — for comparison

```
1: Initialize  $\hat{Q}(s, a) \leftarrow 0$  for all  $s, a$ 
2: for episode = 1, 2, ... do
3:   Generate a full episode  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$  using  $\varepsilon$ -greedy w.r.t.  $\hat{Q}$ 
4:    $G \leftarrow 0$ 
5:   for  $t = T - 1, T - 2, \dots, 0$  do
6:      $G \leftarrow R_{t+1} + \gamma G$ 
7:      $\hat{Q}(S_t, A_t) \leftarrow \hat{Q}(S_t, A_t) + \alpha(G - \hat{Q}(S_t, A_t))$ 
8:   end for
9: end for
```

5 Convergence remarks

- Tabular Sarsa converges to Q^* with probability 1 under the usual stochastic-approximation step-size conditions ($\sum_t \alpha_t = \infty$, $\sum_t \alpha_t^2 < \infty$) together with GLIE exploration (every (s, a) visited infinitely often, policy greedy in the limit).
- n -step Sarsa converges under the same conditions for any fixed finite n ; larger n trades lower bias/faster credit assignment against higher per-update variance.
- MC control converges under GLIE alone (no step-size decay needed if using true sample averages), but only applies to episodic tasks, and only updates after each episode completes — so it can be much slower to propagate information within a long episode than TD-based methods.

6 Reference

Sutton, R. S., & Barto, A. G. *Reinforcement Learning: An Introduction*, 2nd ed., Ch. 5 (Monte Carlo Methods), Ch. 6 (Temporal-Difference Learning, esp. Sarsa), and Ch. 7 (n -step Bootstrapping).